1.1 Introduction

A decision tree is a supervised classification model represented as a tree. Each internal node tests an attribute, each branch represents an outcome of that test, and each leaf gives a class label. The best splitting attribute is selected using a measure such as information gain, gain ratio or Gini index.

1.2 Given loan dataset

No.	Income	Credit history	Employment	Approved
1	High	Good	Salaried	Yes
2	High	Poor	Salaried	No
3	High	Good	Self-employed	Yes
4	Medium	Good	Salaried	Yes
5	Medium	Poor	Self-employed	No
6	Medium	Good	Self-employed	No
7	Medium	Poor	Salaried	No
8	Low	Good	Salaried	Yes
9	Low	Poor	Self-employed	No
10	Low	Poor	Salaried	No
11	Low	Good	Self-employed	No
12	High	Poor	Self-employed	Yes

Class attribute: Approved (Yes/No). Candidate splitting attributes: Income, Credit History and Employment. The tree stops when a node contains records from one class, no useful attributes remain, or a minimum-leaf/pruning condition is reached.

1.3 Entropy of the complete dataset

There are 5 Yes records and 7 No records, giving 12 records in total.

$$\text{Entropy}(S) = -\Sigma\, p_i \log_2(p_i)$$

$$= -(5/12)\log_2(5/12) - (7/12)\log_2(7/12) = 0.9799 \text{ bits}$$

1.4 Information gain at the root

Information Gain(A) = Entropy(S) − Σ ($|S_v|/|S|$) Entropy(S_v).

Attribute	Partition class counts	Weighted entropy	Information gain
Income	High: 3Y,1N; Medium: 1Y,3N; Low: 1Y,3N	0.8113	0.1686
Credit History	Good: 4Y,2N; Poor: 1Y,5N	0.7842	0.1958
Employment	Salaried: 3Y,3N; Self-employed: 2Y,4N	0.9591	0.0208

Credit History has the highest information gain (approximately 0.1958), so it is selected as the root attribute.

1.5 Gini index and gain ratio check

The Gini impurity of the complete data is $1 - (5/12)^2 - (7/12)^2 = 0.4861$. For Credit History, the weighted child Gini is approximately 0.3611, so the Gini reduction is 0.1250. This is greater than the reduction obtained by splitting on Income (approximately 0.1111) or Employment (approximately 0.0278). Therefore, Gini also selects Credit History.

The split information for Credit History is 1 bit because it divides the data into two equal groups of six. Its gain ratio is therefore 0.1958/1 = 0.1958. The alternative criterion does not change the root choice.

1.6 Decision tree and extracted paths

Tree branch / condition	Predicted class
Credit History = Good AND Employment = Salaried	Approved = Yes
Credit History = Good AND Employment = Self-employed AND Income = High	Approved = Yes
Credit History = Good AND Employment = Self-employed AND Income = Medium or Low	Approved = No
Credit History = Poor AND Income = High AND Employment = Self-employed	Approved = Yes
Credit History = Poor AND Income = High AND Employment = Salaried	Approved = No
Credit History = Poor AND Income = Medium or Low	Approved = No

The tree first tests Credit History. For Good credit, Employment separates the salaried records (all Yes) from self-employed records; Income then separates the remaining self-employed records. For Poor credit, Income isolates the High group, and Employment separates the two High-income records. The Medium- and Low-income Poor-credit records are all No.

1.7 High-cardinality attributes and overfitting

An attribute such as Customer_ID may have a different value for nearly every record. A tree can create a separate branch for each ID and appear to classify the training data perfectly, but it will not generalise to new customers. This is overfitting. Such identifiers should normally be excluded unless they have a meaningful predictive interpretation. Pruning, minimum leaf size and cross-validation help control overfitting.

1.8 Weka validation procedure

1. Create a CSV or ARFF file with Income, Credit_History, Employment and Approved as nominal attributes.
2. Open Weka Explorer → Preprocess → Open file and load the 12-record data.
3. Choose Classify → trees → J48. Set Approved as the class attribute.
4. Run J48 on the supplied 12 records, then load and run it on Weka's credit-g (German Credit) data.
5. Change the confidence factor and minimum number of instances per leaf one at a time. Record tree size, leaves and accuracy for each setting.
6. Compare the generated tree with the manual tree. Explain differences caused by tie-breaking, pruning, minimum leaf size or implementation details.

Experiment	Confidence factor	Min instances/leaf	Tree size / leaves	Accuracy
Default settings	Weka default	Weka default	To be recorded	To be recorded
Setting 2	_____	_____	_____	_____
Setting 3	_____	_____	_____	_____

Note: The table is intentionally left for the actual Weka run. Accuracy and tree size depend on the installed Weka version, evaluation mode and selected data.

Q1 Conclusion

Decision-tree induction produces an understandable classification model. In the supplied loan data, Credit History is the best root split according to information gain and Gini reduction. A tree is easy to explain, but it can overfit when it grows too deeply or uses attributes with many distinct values. Pruning and independent testing are therefore important.

Q2 – BAYESIAN CLASSIFICATION AND THE COST OF ASSUMPTIONS

2.1 Introduction

Bayesian classification uses Bayes' theorem to estimate the probability that a record belongs to a class. Naïve Bayes assumes that the input attributes are conditionally independent when the class is known. This assumption simplifies computation and often works well for text classification and other high-dimensional problems.

2.2 Applicant to be classified

Applicant X = (Income = Low, Credit History = Good, Employment = Self-employed).

$$P(\text{Class} \mid X) \propto P(\text{Class}) \times P(\text{Low} \mid \text{Class}) \times P(\text{Good} \mid \text{Class}) \times P(\text{Self-employed} \mid \text{Class})$$

2.3 Prior probabilities

$$P(\text{Yes}) = 5/12 = 0.4167; \quad P(\text{No}) = 7/12 = 0.5833$$

2.4 Conditional probabilities

Condition	Approved = Yes (5 records)	Approved = No (7 records)
Income = Low	1/5 = 0.20	3/7 = 0.4286
Credit History = Good	4/5 = 0.80	2/7 = 0.2857
Employment = Self-employed	2/5 = 0.40	4/7 = 0.5714

2.5 Posterior proportional values

$$\text{Score(Yes)} = (5/12)(1/5)(4/5)(2/5) = 0.02667$$

$$\text{Score(No)} = (7/12)(3/7)(2/7)(4/7) = 0.04082$$

Since Score(No) is greater than Score(Yes), the Naïve Bayes prediction is Approved = No. These are unnormalised posterior scores; they are sufficient to select the larger class score.

2.6 Zero probability and Laplace correction

If a category never occurs with a particular class in the training data, its conditional probability becomes zero. Multiplying by zero makes the entire class score zero, even if the other attributes strongly support that class. Laplace (add-one) smoothing avoids this problem.

For an attribute with k possible values, the smoothed estimate is (count + 1)/(class count + k). For example, if Income has three categories, a zero count for Income = Low in the Yes class becomes (0+1)/(5+3) = 1/8 rather than zero.

In the supplied data, the applicant's listed conditional counts are not zero. Applying add-one smoothing to the three-valued Income and two-valued Credit History and Employment attributes gives:

Conditional value	Yes: add-one	No: add-one
Income = Low	2/8 = 0.2500	4/10 = 0.4000
Credit History = Good	5/7 = 0.7143	3/9 = 0.3333
Employment = Self-employed	3/7 = 0.4286	5/9 = 0.5556

$$\text{Smoothed Score(Yes)} = (5/12)(2/8)(5/7)(3/7) \approx 0.01276$$

$$\text{Smoothed Score(No)} = (7/12)(4/10)(3/9)(5/9) \approx 0.02160$$

The predicted class remains No after smoothing.

2.7 Weka verification and correlated attributes

In Weka, load the loan dataset, select Classify → bayes → NaiveBayes, choose Approved as the class and run the classifier. Inspect the model output and compare the class priors and conditional distributions with the manual table. Small differences may occur because of Weka's probability estimator and smoothing settings.

To study correlated attributes, construct or select a dataset containing two strongly related features, for example height in centimetres and height in inches. Run Naïve Bayes and a decision tree with the same train/test split or cross-validation folds. Record accuracy, precision, recall and confusion matrices. The independence assumption is violated because the two height variables carry nearly duplicate information.

Model	Accuracy	Precision	Recall	Observation
Naïve Bayes	To be measured	To be measured	To be measured	Record actual Weka/Python result
Decision Tree	To be measured	To be measured	To be measured	Record actual Weka/Python result

Naïve Bayes may still rank classes usefully because classification depends on comparing class scores. Even when the probability estimates are not perfectly calibrated, the relative scores can preserve useful information. It is often fast, simple and effective with limited training data. A decision tree may be preferred when interactions between attributes matter or when explicit decision paths are required.

Q2 Conclusion

Naïve Bayes classifies the unseen applicant as No, both before and after Laplace smoothing. It is efficient and provides probability-based class scores, but correlated attributes can make its probability estimates unreliable. The choice between Naïve Bayes and a decision tree should depend on data characteristics, predictive performance and the need for interpretability.

Q3 – RULE-BASED AND ASSOCIATIVE CLASSIFICATION

3.1 Introduction

Rule-based classification represents knowledge using IF–THEN statements. The IF part contains conditions and the THEN part gives a class. A decision tree can be converted into rules by creating one rule for every root-to-leaf path. Associative classification discovers class-association rules from frequent itemsets using support and confidence.

3.2 Rules extracted from Q1

Rule	IF condition	THEN	Coverage	Rule accuracy
R1	Credit = Good AND Employment = Salaried	Yes	3	3/3 = 100%
R2	Credit = Good AND Employment = Self-employed AND Income = High	Yes	1	1/1 = 100%
R3	Credit = Good AND Employment = Self-employed AND Income ∈ {Medium, Low}	No	2	2/2 = 100%
R4	Credit = Poor AND Income = High AND Employment = Self-employed	Yes	1	1/1 = 100%
R5	Credit = Poor AND Income = High AND Employment = Salaried	No	1	1/1 = 100%

R6	Credit = Poor AND Income ∈ {Medium, Low}	No	4	4/4 = 100%

Coverage is the number of records satisfying the rule antecedent. Rule accuracy is the proportion of covered records correctly assigned to the rule's predicted class. The six rules cover all 12 records and classify the supplied training data correctly.

3.3 Rule conflict resolution

A conflict occurs when more than one rule matches the same record but predicts different classes. One approach is size ordering: prefer the rule with more specific conditions. Another is rule ordering: evaluate rules in a predefined priority sequence and use the first matching rule. For example, a broad rule such as IF Credit = Good THEN Yes would conflict with the more specific rule IF Credit = Good AND Employment = Self-employed AND Income = Medium THEN No. The more specific rule should take priority because it uses additional evidence.

3.4 Sequential covering

Sequential covering learns one rule at a time. It finds a rule that covers a useful group of training records, adds the rule to the rule set, and removes or marks the positive examples already covered. It repeats the process until the stopping condition is met. A final default rule may classify records not covered by any learned rule.

3.5 Associative classification

Associative classification mines class-association rules of the form X → class. Support measures how frequently X and the class occur together in the complete dataset. Confidence measures how often the class occurs among records satisfying X.

$$\text{Support}(X \rightarrow C) = \text{count}(X \text{ and } C) \,/\, \text{total records}$$

$$\text{Confidence}(X \rightarrow C) = \text{count}(X \text{ and } C) \,/\, \text{count}(X)$$

Minimum support = 25% of 12 = 3 records. Minimum confidence = 70%. From the supplied tuples, the rule Income = High → Approved = Yes occurs in 3 of 12 records, so support = 25%. Among the four High-income records, three are approved, so confidence = 3/4 = 75%. It satisfies both thresholds.

Class-association rule predicting Yes	**Support**	**Confidence**	**Meets thresholds?**
Income = High → Approved = Yes	3/12 = 25%	3/4 = 75%	Yes

The other single-attribute values predicting Yes do not meet both thresholds. For example, Credit = Good has confidence 4/6 = 66.7%, below 70%.

3.6 Weka JRip/PART comparison

Load the Weka credit-g dataset and select Classify → rules → JRip (or PART). Run the classifier using the same evaluation method used for J48. Record the generated rules, number of rules, accuracy and confusion matrix. Compare them with J48: JRip learns an ordered rule set, while PART builds partial decision trees and converts selected leaves into rules.

Classifier	**Number of rules/tree leaves**	**Accuracy**	**Interpretation**
J48	To be recorded	To be recorded	Tree paths can be converted to rules
JRip or PART	To be recorded	To be recorded	Rule ordering and pruning affect output

3.7 Recommended representation for loan rejection explanations

For a loan officer who must explain every rejection, a pruned decision tree or a carefully ordered rule set is suitable. A decision tree provides a clear route from the root to a decision. Rules are particularly convenient because each rejection can be explained by quoting the exact conditions that matched. Associative rules are useful for discovering frequent patterns, but they may produce many overlapping rules and may not provide a complete explanation for every individual application.

Q3 Conclusion

Rules make classification decisions explicit and easy to communicate. The tree-derived rules cover all 12 loan records. Under the specified support and confidence thresholds, Income = High → Approved = Yes is a qualifying class-association rule. For individual loan explanations, a pruned tree or ordered rule list is recommended.

Q4 – BACKPROPAGATION: WHAT DOES THE NETWORK LEARN?

4.1 Introduction

A feed-forward neural network contains input, hidden and output layers. Backpropagation trains the network by calculating the output error and propagating that error backwards to adjust weights and biases. The sigmoid activation function maps any real input to a value between zero and one.

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

4.2 Given network

Inputs: x1 = 1, x2 = 0; target t = 1; learning rate η = 0.9. Hidden units: h1 and h2. Output unit: o.

Weight / bias	Initial value
x1 → h1	0.2
x2 → h1	0.4
x1 → h2	−0.3
x2 → h2	0.1
h1 → o	−0.3
h2 → o	−0.2
Bias h1	−0.4
Bias h2	0.2
Bias output	0.1

4.3 Forward pass

$$\text{net}(h1) = (1)(0.2) + (0)(0.4) - 0.4 = -0.2000; \quad h1 = \sigma(\text{net}) = 0.4502$$

$$\text{net}(h2) = (1)(-0.3) + (0)(0.1) + 0.2 = -0.1000; \quad h2 = \sigma(\text{net}) = 0.4750$$

$$\text{net}(o) = (h1)(-0.3) + (h2)(-0.2) + 0.1 = -0.1301; \quad o = 0.4675$$

The initial output is 0.4675, while the target is 1. The initial squared-error loss using $E = \frac{1}{2}(t - o)^2$ is 0.141761.

4.4 Backward pass

For the output node, $\delta o = (t - o)o(1 - o)$. For a hidden node, $\delta h = h(1 - h) \times \delta o \times$ (weight from that hidden node to the output).

$$\delta o = (1 - 0.4675)(0.4675)(1 - 0.4675) = 0.132556$$

$$\delta h1 = h1(1-h1)\delta o(-0.3) = -0.009843$$

$$\delta h2 = h2(1-h2)\delta o(-0.2) = -0.006611$$

4.5 Weight and bias updates

The update rule is $w(new) = w(old) + \eta \times \delta \times input$. A bias is updated as $b(new) = b(old) + \eta \times \delta$.

Parameter	Updated value
x1 → h1	0.191141
x2 → h1	0.400000
Bias h1	-0.408859
x1 → h2	-0.305950
x2 → h2	0.100000
Bias h2	0.194050
h1 → o	-0.246295
h2 → o	-0.143330
Bias output	0.219300

After updating all parameters, the recomputed hidden outputs are h1 = 0.4458 and h2 = 0.4721. The new output is 0.5105, and the new loss is 0.119825.

The error has decreased.

4.6 Why small random initial weights are used

Small random values break symmetry between hidden units. If all weights are initialized to exactly the same value, hidden units may produce identical outputs and receive identical updates. They may therefore learn the same feature instead of developing different useful representations. Very large initial weights can also push sigmoid units into saturation, where gradients become very small.

4.7 Weka/Python experiment plan

Train a MultilayerPerceptron in Weka or an equivalent neural network in Python on a real dataset. Change one setting at a time: learning rate, number of hidden units and number of epochs. Use the same training/test split or cross-validation folds for fair comparison.

Run	Learning rate	Hidden units	Epochs	Training accuracy	Test accuracy	Training time
1	Default	Default	Default	To be measured	To be measured	To be measured
2	Low	Same	Same	To be measured	To be measured	To be measured
3	High	Same	Same	To be measured	To be measured	To be measured
4	Same	More	Same	To be measured	To be measured	To be measured
5	Same	Same	More	To be measured	To be measured	To be measured

Overfitting is indicated when training accuracy continues to improve but test accuracy stops improving or decreases. Slow or unstable training may occur when the learning rate is poorly chosen, the data are not scaled, or the network is too large.

Q4 Conclusion

Backpropagation adjusts network parameters in the direction that reduces prediction error. In the given one-step example, the output error is propagated to both hidden units and all weights and biases are updated. Neural networks can model complex relationships, but they require parameter tuning and are less interpretable than decision trees.

Q5 – SUPPORT VECTOR MACHINES: MARGIN, KERNELS AND SCALING

5.1 Introduction

A Support Vector Machine (SVM) is a supervised learning method that finds a decision boundary separating classes. The maximum-margin hyperplane is the boundary that maximises the distance to the nearest training points from each class. These nearest points are called support vectors.

5.2 Given points

Class	Coordinates
+1	(3,3), (4,3), (4,4)
−1	(1,1), (1,2), (2,1)

The positive points have x1 + x2 values of 6, 7 and 8, while the negative points have sums of 2 or 3. The closest class boundaries are therefore x1 + x2 = 6 and x1 + x2 = 3.

5.3 Separating hyperplane and support vectors

$$x1 + x2 - 4.5 = 0$$

A normal vector is w = (1,1) and b = −4.5. The support vectors are (3,3) from class +1 and (2,1), (1,2) from class −1. They are the points closest to the separating hyperplane and determine its position.

5.4 Margin width

The perpendicular distance between two parallel lines $w \cdot x + b1 = 0$ and $w \cdot x + b2 = 0$ is $|b1 - b2|/||w||$. Here the support-vector boundary lines are x1+x2−6=0 and x1+x2−3=0.

$$\text{Margin width} = (6 - 3) / \sqrt{(1^2 + 1^2)} = 3/\sqrt{2} \approx 2.121 \text{ units}$$

The distance from the central hyperplane to either support-vector boundary is half of this, approximately 1.061 units.

5.5 Removing points, soft margin and kernels

- Removing a non-support-vector point normally does not change the maximum-margin hyperplane, provided it does not become a new closest point.
- Removing a support vector can change the hyperplane and margin because support vectors directly determine the boundary.
- The soft-margin parameter C controls the penalty for classification errors. A large C penalises errors strongly and may create a narrower, more complex boundary; a small C allows more violations and stronger regularisation.
- Kernels allow SVMs to model non-linear boundaries by computing similarity in a transformed feature space. Common kernels are linear, polynomial and radial basis function (RBF).

5.6 Kernel and scaling experiments

Use Weka SMO or Python SVC on a non-linear dataset such as concentric circles. Compare linear, polynomial and RBF kernels. Change C and the kernel parameters, and record accuracy and number of support vectors. Repeat the experiment before and after feature scaling.

Kernel / setting	C and kernel parameters	Accuracy	Support vectors	Notes
Linear	Record settings	To be measured	To be measured	Baseline
Polynomial	Record degree and C	To be measured	To be measured	Captures polynomial boundaries
RBF	Record gamma and C	To be measured	To be measured	Flexible non-linear boundary
RBF after scaling	Record gamma and C	To be measured	To be measured	Compare fairly with unscaled data

Scaling is important because SVMs depend on distances and dot products. If one feature is measured in thousands while another ranges from zero to one, the large-scale feature can dominate the margin. Standardisation or min–max scaling should be fitted using training data only and then applied to test data.

5.7 When to choose SVM

SVM is useful for medium-sized datasets with clear margins, high-dimensional features and potentially non-linear class boundaries. Practical limitations include parameter selection, sensitivity to feature scaling, longer training time on large datasets and limited direct interpretability compared with a small decision tree.

Q5 Conclusion

For the supplied six points, $x1 + x2 - 4.5 = 0$ is a separating maximum-margin hyperplane, with three support vectors and a margin width of approximately 2.121 units. Kernels extend SVM to non-linear data, while feature scaling and parameter tuning are essential for reliable performance.

Q6 – EVALUATING CLASSIFIERS AND PREDICTORS

6.1 Introduction

Classifier evaluation measures how well a model assigns records to classes. Accuracy alone can be misleading when classes are imbalanced. Precision, recall, specificity, F-measure, ROC analysis and cross-validation provide additional information. Prediction models for numerical values are evaluated using measures such as MAE, MSE and RMSE.

6.2 Confusion matrix

Given: TP = 40, FN = 20, FP = 30 and TN = 910. Total transactions = 1,000.

	Predicted fraud	**Predicted not fraud**	**Total**
Actual fraud	TP = 40	FN = 20	60
Actual not fraud	FP = 30	TN = 910	940
Total	70	930	1000

6.3 Classification metrics

Measure	**Formula**	**Calculation**	**Result**
Accuracy	(TP+TN)/N	(40+910)/1000	95.00%
Error rate	(FP+FN)/N	(30+20)/1000	5.00%
Sensitivity / Recall	TP/(TP+FN)	40/60	66.67%
Specificity	TN/(TN+FP)	910/940	96.81%
Precision	TP/(TP+FP)	40/70	57.14%
F-measure / F1	2TP/(2TP+FP+FN)	80/130	61.54%

Recall measures the proportion of actual fraud cases detected. Precision measures the proportion of alerts that are truly fraud. In fraud detection, both are important, but the preferred balance depends on the cost of missed fraud and the cost of investigating false alerts.

6.4 Accuracy under class imbalance

If the classifier predicts “not fraud” for every transaction, it correctly classifies all 940 non-fraud transactions and misses all 60 fraud transactions. Its accuracy is 940/1000 = 94%, but its fraud recall is 0%. This demonstrates why accuracy alone is insufficient for an imbalanced dataset.

6.5 Prediction error measures

Actual values = {12, 15, 20, 25}; predicted values = {10, 18, 19, 30}. Errors (actual − predicted) are 2, −3, 1 and −5.

$$\text{MAE} = (|2| + |-3| + |1| + |-5|) / 4 = 11/4 = 2.75$$

$$\text{MSE} = (2^2 + (-3)^2 + 1^2 + (-5)^2) / 4 = 39/4 = 9.75$$

$$\text{RMSE} = \sqrt{9.75} \approx 3.122$$

MAE gives the average absolute error in the original unit and is less sensitive to large errors. RMSE penalises large errors more strongly because errors are squared before averaging. RMSE is preferred when large prediction mistakes are especially undesirable.

6.6 ROC curve

A Receiver Operating Characteristic (ROC) curve plots the true positive rate (sensitivity) against the false positive rate (1 − specificity) at different classification thresholds. A curve closer to the top-left corner indicates better discrimination. The area under the ROC curve (AUC) summarises ranking performance: 0.5 represents chance-level ranking and 1.0 represents perfect ranking. For highly imbalanced problems, the precision–recall curve should also be examined.

6.7 Holdout, cross-validation and bootstrap

Method	How it works	Strength	Limitation
Holdout	Split data once into training and test sets	Simple and fast	Result depends on one split
10-fold cross-validation	Train on 9 folds and test on the remaining fold; repeat 10 times	Uses data efficiently and reduces split dependence	Requires multiple model fits
Bootstrap (.632)	Sample training records with replacement; evaluate on records left out of each sample and combine estimates	Useful for estimating variability, especially with limited data	More complex; bootstrap estimate has its own bias

Repeat holdout with at least five different random splits and report each accuracy and the mean and standard deviation. For a fair comparison of J48, Naïve Bayes and SMO, use identical folds and paired evaluation. A difference should be interpreted with its variation or a suitable statistical comparison, not from one accuracy value alone.

Split / fold	J48 accuracy	Naïve Bayes accuracy	SMO accuracy
1	To be measured	To be measured	To be measured
2	To be measured	To be measured	To be measured
3	To be measured	To be measured	To be measured
4	To be measured	To be measured	To be measured
5	To be measured	To be measured	To be measured

Q6 Conclusion

The fraud classifier has 95% accuracy, 66.67% recall and 57.14% precision. A trivial classifier can reach 94% accuracy while detecting no fraud, proving that class-sensitive measures are essential. Evaluation should use repeated or cross-validated estimates and include metrics relevant to the real cost of errors.

Q7 – DESIGN A CLASSIFICATION SOLUTION FROM FIRST PRINCIPLES

7.1 Problem statement

A hospital wants to identify patients likely to be readmitted within 30 days. Only 6% of patients are readmitted. Missing a high-risk patient (false negative) is ten times more costly than flagging a patient who is not at risk (false positive). Doctors must understand the reason for every alert. Available attributes are age, diagnosis group, length of stay, prior admissions, laboratory results and discharge medications; some values are missing.

7.2 Stage 1 – Independent design proposals

Student A – initial proposal (complete independently before discussion):

Design element	**Student A proposal**
Classification technique	_____
Preprocessing / missing values	_____
Imbalance handling	_____
Evaluation metric / estimation	_____
Explanation to doctors	_____

Student B – initial proposal (complete independently before discussion):

Design element	**Student B proposal**
Classification technique	_____
Preprocessing / missing values	_____
Imbalance handling	_____
Evaluation metric / estimation	_____
Explanation to doctors	_____

The two initial proposals should be retained as evidence of independent thinking. The following sections provide a reasoned example of a joint final design.

7.3 Stage 2 – Compare the designs

Aspect	**Where proposals agree / differ**	**Decision for final design**
Model choice	To be completed by the pair	Select a model balancing recall and interpretability
Missing values	To be completed by the pair	Use train-fitted imputation and missingness indicators where useful
Imbalance	To be completed by the pair	Use cost-sensitive learning and threshold tuning
Evaluation	To be completed by the pair	Use stratified cross-validation and cost-sensitive metrics
Explainability	To be completed by the pair	Provide reasons for each alert

7.4 Stage 3 – Joint final design

A practical solution is to begin with an interpretable baseline such as a pruned decision tree or logistic regression, and compare it with a stronger ensemble such as a class-weighted Random Forest. The final deployed model should be selected using cross-validated recall, precision and expected cost, subject to the requirement that doctors can understand each alert.

Preprocessing

- Remove duplicate records and check impossible or inconsistent values, such as negative length of stay or invalid laboratory measurements.
- Split the data into training and test sets before fitting imputers, scalers, feature selection or resampling. Use stratified splitting to preserve the 6% readmission rate.
- Impute numerical missing values using training-set medians. Impute categorical values using the most frequent category or an explicit Unknown category.
- Consider missingness indicators because a missing laboratory test may itself reflect a clinical workflow decision.
- Encode diagnosis group and discharge medication categories using an appropriate categorical encoding. Scale numerical variables for models that require it.
- Handle imbalance with class weights or cost-sensitive learning. If resampling is used, apply it only within training folds, never to validation or test data.
- Choose the decision threshold using the 10:1 false-negative to false-positive cost ratio and validation data.

Model and interpretation

A shallow/pruned decision tree is a strong interpretable candidate because it can show explicit paths such as: IF prior admissions are high AND length of stay is long THEN flag for review. A class-weighted logistic regression can provide a second interpretable baseline. A Random Forest may improve predictive performance, but individual explanations should be provided using feature contributions or local explanation methods and checked by clinicians. Explanations should support, not replace, clinical judgement.

7.5 Stage 4 – Analytical validation and expected cost

For a 10:1 cost ratio, define the total error cost as:

$$\text{Total cost} = 10 \times FN + 1 \times FP$$

Consider two illustrative decision thresholds on 1,000 patients, including 60 actual readmissions and 940 non-readmissions. These matrices are examples to demonstrate the calculation; they are not measured model results.

Threshold	TP	FN	FP	TN	Total cost = 10FN + FP
A	45	15	100	840	10(15)+100 = 250
B	55	5	180	760	10(5)+180 = 230

Threshold B has more false positives, but it reduces false negatives by ten. Since each false negative costs ten times as much as a false positive, the total illustrative cost falls from 250 to 230. This shows why the threshold should not automatically be 0.5. The final threshold must be selected using validation predictions and the hospital's agreed cost assumptions.

Accuracy is inappropriate as the sole selection measure because 94% of patients are not readmitted. A model that predicts no readmissions for anyone could achieve 94% accuracy while missing every high-risk patient. Recall, precision, PR-AUC, ROC-AUC and total cost provide more meaningful evidence.

7.6 Experimental validation

Student B should implement the final design and a simple baseline in Weka or Python using a public analogue dataset. Use stratified cross-validation and report recall, precision, ROC-AUC, PR-AUC where possible, confusion matrix and total cost. Compare the model with and without class weighting or resampling.

Experiment	Imbalance method	Recall	Precision	ROC-AUC	Total cost
Baseline	None	To be measured	To be measured	To be measured	To be measured
Model 2	Class weights	To be measured	To be measured	To be measured	To be measured

Model 3	Training-fold resampling	To be measured	To be measured	To be measured	To be measured

Note: Use a public dataset that matches the readmission task as closely as possible. A diabetes readmission dataset is one possible analogue, but its label definition and population must be checked before interpreting results.

7.7 Strengths, limitations and failure conditions

- Strength: cost-sensitive learning gives greater importance to missed high-risk patients.
- Strength: a shallow decision tree or logistic regression can provide understandable explanations.
- Limitation: improving recall may increase false-positive alerts and clinician workload.
- Limitation: missing or inaccurate clinical records can reduce reliability.
- Failure condition: changes in clinical practice, patient population or hospital workflow can cause data drift.
- Failure condition: a model trained at one hospital may not generalise to another hospital without external validation.
- Failure condition: explanations may be misleading if the model uses biased or non-causal correlations.

7.8 Stage 5 – Cross-defence

Student A should explain which experimental evidence supports the group's design, including cross-validation results, recall, precision and total cost. Student B should explain how the 10:1 cost ratio is applied to the confusion matrix and how the threshold was selected. Both students should be prepared to defend the partner's work.

7.9 Stage 6 – Joint evaluation

Standard accuracy-driven classifiers struggle because the positive class is rare and the cost of the two error types is unequal. The design may sacrifice some precision and increase the number of alerts to reduce costly false negatives. The final model should be monitored after deployment, reviewed by clinicians and periodically validated on new data.

Before choosing any classifier, a data miner should examine the number of records, number and type of attributes, missing values, class balance, noise, outliers, feature scales, relationships between attributes, data leakage risks, interpretability requirements, error costs, available computing resources and the method by which future performance will be monitored.

Q7 Conclusion

The hospital readmission problem requires a cost-sensitive and interpretable classification system. A model should be selected using stratified validation, recall, precision, AUC and the explicit 10:1 error-cost measure rather than accuracy alone. A pruned decision tree or logistic regression provides a transparent baseline, while more complex models can be compared if reliable explanations are available.

OVERALL CONCLUSION

This assignment examined the main classification and prediction techniques used in data mining. Decision trees select informative attributes and produce understandable paths. Naïve Bayes applies probability theory and can classify efficiently, although correlated features may violate its independence assumption. Rule-based and associative classification express patterns as IF–THEN relationships. Backpropagation trains neural networks by propagating errors through layers. SVM identifies a maximum-margin boundary and can model non-linear patterns through kernels.

The evaluation section demonstrated that accuracy alone is not sufficient for imbalanced data. Recall, precision, specificity, F-measure, ROC analysis, cross-validation and prediction-error measures provide a more complete view of model performance. Finally, the hospital case showed that the classifier must reflect the real-world cost of errors and provide explanations that support human decision-making.

A successful data-mining solution is not chosen only because an algorithm is popular. It must be selected after examining the dataset, the target problem, data quality, class distribution, cost of errors, interpretability needs and the reliability of experimental evaluation.

REFERENCES

7. Han, J., Kamber, M., and Pei, J. Data Mining: Concepts and Techniques. 3rd Edition, Morgan Kaufmann.
8. Tan, P.-N., Steinbach, M., and Kumar, V. Introduction to Data Mining. Pearson.
9. Witten, I. H., Frank, E., Hall, M. A., and Pal, C. J. Data Mining: Practical Machine Learning Tools and Techniques. Morgan Kaufmann.
10. WEKA – Waikato Environment for Knowledge Analysis. Official documentation and Explorer user guide. https://www.cs.waikato.ac.nz/ml/weka/
11. Scikit-learn User Guide – Decision Trees, Naïve Bayes, Neural Networks, Support Vector Machines and Model Evaluation. https://scikit-learn.org/stable/user_guide.html
12. Course assignment sheet: CSA16 Data Warehousing and Data Mining, Assignment 3 – CO3.

APPENDIX A – CONTRIBUTION RECORD

Question	**Student A – Primary contribution**	**Student B – Primary contribution**	**Joint work**
Q1	_____	_____	_____
Q2	_____	_____	_____
Q3	_____	_____	_____
Q4	_____	_____	_____
Q5	_____	_____	_____
Q6	_____	_____	_____
Q7	_____	_____	_____

The contribution record must reflect the actual work completed. Primary contribution does not mean exclusive responsibility. Both students remain accountable for the complete assignment.